

REPORTE ACADÉMICO: PROYECTO DE SISTEMA DE RIEGO DOMÉSTICO INTELIGENTE

Autor: Adrián Fernando Cuevas Solís
 Afiliación: Universidad Autónoma de Zacatecas (UAZ)
 Materia: Internet de las Cosas (IoT) - Noveno Semestre
 Fecha: 10 de diciembre de 2025

1. NOMBRE DEL PROYECTO	1
2. INTRODUCCIÓN	1
3. MARCO TEÓRICO	1
3.1. Fundamentos de IoT y su Aplicación	1
3.2. Plataforma de Hardware: ESP32	2
3.3. Protocolos de Comunicación: MQTT y BLE	2
3.4. Entorno de Programación: MicroPython	2
3.5. Sensores y Principios de Operación	2
3.6. Plataforma de Visualización: Node-RED	3
4. DESARROLLO	3
4.1. Arquitectura del Sistema	3
4.2. Implementación Hardware	3
4.3. Implementación Software y Desafíos Técnicos	4
4.4. Comunicaciones MQTT	4
4.5. Dashboard Node-RED	5
4.6. Estrategias de Mitigación y Fallo Final	5
5. RESULTADOS	5
5.1. Limitaciones Técnicas Identificadas	5
5.2. Evaluación de Soluciones Intentadas	6
5.3. Lecciones Aprendidas	6
6. CONCLUSIONES	6
6.1. Logros Parciales	6
6.2. Limitaciones Críticas Identificadas	7
6.3. Recomendaciones para Implementaciones Futuras	7
6.4. Valor Educativo	7
7. REFERENCIAS	8

1. NOMBRE DEL PROYECTO

Sistema Autónomo de Gestión Hídrica para Horticultura Doméstica con Monitoreo Remoto: Una Implementación con ESP32, MicroPython y Node-RED.

2. INTRODUCCIÓN

La gestión eficiente del agua en la jardinería doméstica representa un desafío significativo, particularmente para grupos poblacionales como personas mayores o individuos con frecuentes ausencias de su domicilio. El riego tradicional, basado en rutinas fijas o apreciaciones subjetivas, conduce frecuentemente al desperdicio de agua o al estrés hídrico de las plantas. Este problema se agrava en un contexto global donde la escasez hídrica demanda soluciones de precisión y conservación.

La tecnología del Internet de las Cosas (IoT) emerge como un paradigma idóneo para abordar esta problemática, permitiendo la monitorización en tiempo real de variables ambientales y la automatización de respuestas adaptativas. El proyecto se enmarca en la intersección de la domótica (automatización del hogar) asistencial y la agricultura de precisión; aplicando principios de IoT para crear un prototipo funcional que no solo automatice una tarea, sino que optimice el recurso hídrico. Como se ha observado en otros contextos, la aplicación de IoT para la automatización y el análisis de datos es una palanca clave para la eficiencia.

Objetivo General: Diseñar, implementar y validar un sistema automatizado de riego doméstico, inteligente y conectado, que asista a usuarios vulnerables optimizando el consumo de agua.

Objetivos Específicos:

- Integrar una red de sensores (humedad de suelo, temperatura ambiental y nivel de agua) con un microcontrolador ESP32.
- Desarrollar una lógica de control programada en MicroPython que active riego basado en datos multisensoriales y restricciones horarias.
- Implementar un dashboard de supervisión y control remoto utilizando Node-RED y el protocolo de comunicación MQTT.
- Agregar una función de cambio de colores RGB mediante BLE para personas con problemas visuales.
- Evaluar la eficiencia hídrica del sistema y la estabilidad de sus componentes en un ciclo de operación continuo.

3. MARCO TEÓRICO

3.1. Fundamentos de IoT y su Aplicación

El Internet de las Cosas (IoT) constituye una red de objetos físicos equipados con sensores, software y conectividad para intercambiar datos con otros dispositivos y sistemas a través de Internet. Su arquitectura típica incluye dispositivos finales (sensores/actuadores), comunicaciones locales, *parsers*, servidores y aplicaciones en la nube. En el ámbito de la eficiencia de recursos, el IoT actúa mediante la automatización inteligente, el mantenimiento

predictivo y la optimización basada en datos analíticos. Proyectos que integran IoT en educación y solución de problemas cotidianos demuestran su potencial para fortalecer competencias técnicas y ofrecer soluciones innovadoras.

3.2. Plataforma de Hardware: ESP32

El microcontrolador ESP32 fue seleccionado como el núcleo de procesamiento del sistema debido a sus características técnicas superiores para aplicaciones IoT: doble núcleo Xtensa LX6, conectividad Wi-Fi y Bluetooth Low Energy (BLE) integradas, bajo consumo energético y una amplia gama de periféricos (ADC, DAC, interfaces SPI/I2C). Estas capacidades lo posicionan por encima de alternativas como Arduino UNO, permitiendo manejar concurrentemente la lectura de sensores, la lógica de control, la comunicación MQTT y un servicio BLE, todo dentro de un único dispositivo.

3.3. Protocolos de Comunicación: MQTT y BLE

Para la comunicación de red, se eligió el protocolo MQTT (Message Queuing Telemetry Transport). Este sigue un modelo de publicación/suscripción (pub/sub) que es ligero en ancho de banda y consumo energético, ideal para dispositivos embebidos con recursos limitados. Opera a través de un *broker* central que gestiona los *topics*, desacoplando a los dispositivos que publican datos (por ejemplo, el ESP32) de aquellos que los consumen (por ejemplo, el dashboard Node-RED).

Complementariamente, se utiliza Bluetooth Low Energy (BLE) para configuraciones de corto alcance. BLE es óptimo para crear una interfaz directa y de bajo consumo entre el dispositivo y una aplicación móvil, utilizada en este proyecto para configurar los colores del LED RGB de manera local sin necesidad de conectividad a Internet.

3.4. Entorno de Programación: MicroPython

MicroPython es una implementación eficiente del lenguaje Python 3 optimizada para ejecutarse en microcontroladores. Fue seleccionado sobre alternativas como el Arduino IDE (C++) por su sintaxis clara y productiva, su entorno interactivo (REPL) que facilita la depuración, y su soporte nativo para funcionalidades de red esenciales para IoT, como clientes MQTT y sockets Wi-Fi.

3.5. Sensores y Principios de Operación

- Sensor de Humedad de Suelo Capacitivo: Mide el contenido volumétrico de agua del suelo mediante la detección de cambios en la constante dieléctrica, la cual varía con la humedad. Es anticorrosivo y proporciona una medición más estable y duradera que los sensores resistivos.
- Sensor DHT22: Mide temperatura y humedad relativa ambiental. Utiliza un sensor capacitivo para la humedad y un termómetro para la temperatura, enviando los datos digitalmente.
- Sensor de Nivel por Flotador (Horizontal Float Switch): Es un interruptor mecánico donde un flotador acciona un contacto *reed* (magnético) al alcanzar un nivel de agua específico, indicando estados discretos como "lleno" o "vacío".

3.6. Plataforma de Visualización: Node-RED

Node-RED es una herramienta de programación visual basada en flujos (*flows*) para conectar dispositivos de hardware, APIs y servicios en línea. Es ideal para prototipar rápidamente dashboards IoT, ya que permite mediante nodos predefinidos suscribirse a topics MQTT, procesar datos, crear interfaces de usuario gráficas (UI) y generar comandos de control sin escribir código complejo del lado del servidor.

4. DESARROLLO

4.1. Arquitectura del Sistema

El sistema sigue una arquitectura híbrida y en capas que integra componentes físicos y lógicos para lograr un funcionamiento autónomo y supervisable.

Componentes Principales y su Función:

- **Capa Física/Sensorial:** Incluye los sensores de humedad de suelo, DHT22 y de nivel de agua, junto con los actuadores (electroválvulas y LED RGB). Su función es interactuar con el entorno.
- **Capa de Control y Adquisición (ESP32):** Es el cerebro embebido. Lee los sensores, ejecuta la lógica de decisión en MicroPython y comanda los actuadores. También gestiona todas las comunicaciones.
- **Capa de Comunicación:** Utiliza Wi-Fi para MQTT (conexión a largo alcance/Internet) y BLE para configuración local. El ESP32 publica datos y se suscribe a comandos a través de un broker MQTT público (broker.emqx.io).
- **Capa de Aplicación y Visualización (Node-RED):** Alberga el *broker* MQTT y el servidor web del dashboard. Procesa los datos recibidos, los presenta en una interfaz gráfica y permite el control manual remoto.
- **Capa de Usuario:** Los usuarios finales interactúan con el sistema a través del dashboard web o, de manera local, mediante una app BLE para configurar el LED.

4.2. Implementación Hardware

El esquema de conexiones se diseñó considerando la gestión de potencia y la protección de los pines del ESP32.

- **Alimentación:** El sistema se divide en dos dominios de voltaje. El ESP32, sensores DHT22 y capacitivos funcionan a 3.3V. Las electroválvulas (5V DC) son controladas a través de módulos de relé independientes, los cuales son activados por los pines GPIO del ESP32 (a 3.3V) pero están eléctricamente aislados del circuito de potencia de las válvulas.
- **Conexión de Sensores:** Los sensores capacitivos y el DHT22 se conectan a pines de entrada analógica (ADC) y digital, respectivamente. Los sensores de nivel de flotador, al ser interruptores simples, se conectan a pines digitales con resistencias *pull-up* internas o externas.
- **LED RGB:** Se conecta mediante tres pines PWM del ESP32 (uno por color) con resistencias limitadoras de corriente en serie.

4.3. Implementación Software y Desafíos Técnicos

El desarrollo del firmware en MicroPython enfrentó desafíos técnicos significativos relacionados con las limitaciones de recursos del ESP32. Inicialmente, se diseñó una arquitectura de software que integraba:

- Lectura periódica de sensores (cada 15 minutos)
- Servicio BLE activo para configuración local del LED RGB
- Cliente MQTT permanente para comunicación con Node-RED
- Lógica de control basada en múltiples variables

Sin embargo, durante las pruebas se identificó que el ESP32 no podía manejar concurrentemente las pilas de comunicación BLE y MQTT junto con las operaciones de sensores y actuadores. El microcontrolador presentaba limitaciones críticas de memoria RAM y flash al intentar ejecutar ambos protocolos simultáneamente.

4.4. Comunicaciones MQTT

La estructura de topics MQTT sigue una convención jerárquica para una organización clara:

afcs_fito	esp32	proyRiego	salidas	riego	p1,p2,p3,p4		
				estados	contenedor		
					tempAmb		
					plantas	p1,p2,p3,p4	
			entradas	estados	contenedor		
					tempAmb		
					plantas	hum	p1,p2,p3,p4
			reigo	p1,p2,p3,p4			
			registros				
			color	led_medio			
				led_bajo			

El ESP32 publica datos en los topics de sensores y estado con QoS (Calidad de Servicio) 0 (máximo una vez) por su bajo overhead. Para los comandos de válvula enviados desde el dashboard, se usa QoS 1 (al menos una vez) para garantizar la recepción.

4.5. Dashboard Node-RED

El dashboard se implementó con los siguientes grupos de nodos:

- Entrada MQTT: Nodos suscritos a los *topics* de datos del ESP32.
- Procesamiento: Nodos *function* de JavaScript para formatear datos (ej., convertir valores analógicos a porcentaje).
- Visualización: Nodos de UI (gauges, charts, text displays, leds) organizados en una pestaña web.
- Salida MQTT: Nodos publicadores para enviar comandos (ej., ``on/off` a afcs_fito/esp32/proyRiego/entradas/riego/p1`). Se implementó un bloqueo por software en Node-RED que, tras un comando manual, deshabilita el switch por 1 hora para evitar riegos consecutivos durante labores de mantenimiento.

4.6. Estrategias de Mitigación y Fallo Final

Para abordar estos problemas, se implementaron varias estrategias de mitigación:

1. **Alternancia Temporal (Time-Sharing):** Se diseñó un sistema donde BLE y MQTT se activaban de forma alternada, limpiando la memoria entre cada cambio de contexto.
2. **Sistema de Colas:** Se implementó un buffer en memoria flash para almacenar órdenes MQTT recibidas durante los períodos de actividad BLE, y viceversa.
3. **Optimización de Memoria:** Se redujo al mínimo el uso de librerías y se implementaron funciones manuales de gestión de memoria.

A pesar de estas estrategias, el sistema presentó fallos críticos recurrentes:

- **Errores de Memoria Flash:** Cada cierto tiempo (variable, entre 2-48 horas), el sistema generaba errores internos en la memoria flash que requerían formateo completo.
- **Corrupción de Sistema de Archivos:** En ocasiones, los errores causaban daños irreparables que requerían reinstalación completa de MicroPython y el firmware.
- **Comportamiento Inestable:** La necesidad constante de resetear y reprogramar el dispositivo imposibilitaba su funcionamiento autónomo.

Finalmente, se determinó que las limitaciones de hardware del ESP32 (particularmente la memoria flash de 4MB y RAM limitada) eran insuficientes para la complejidad del sistema propuesto, especialmente con la carga adicional del módulo ``machine`` para control preciso de temporizadores y periféricos.

5. RESULTADOS

5.1. Limitaciones Técnicas Identificadas

El proyecto no logró implementarse completamente debido a limitaciones fundamentales del hardware seleccionado. Los principales hallazgos fueron:

1. **Incompatibilidad de Protocolos:** El ESP32 demostró incapacidad para manejar simultáneamente las pilas de protocolo BLE y MQTT en MicroPython, especialmente bajo cargas periódicas de lectura de sensores y control de actuadores.
2. **Limitaciones de Memoria:** Con solo 4MB de flash y aproximadamente 520KB de RAM (de los cuales solo ~100KB están disponibles para heap en MicroPython), el sistema rápidamente alcanzó sus límites al cargar:
 - a. Librería MQTT (approx. 50KB)
 - b. Librería BLE (approx. 40KB)
 - c. Módulo machine para control de hardware
 - d. Código de aplicación y buffers de datos
3. **Problemas de Estabilidad:** La estrategia de alternancia entre protocolos generaba condiciones de carrera (race conditions) y pérdida de datos, mientras que el acceso frecuente a la memoria flash para almacenamiento temporal aceleraba su degradación.

5.2. Evaluación de Soluciones Intentadas

Se probaron múltiples enfoques para resolver los problemas:

1. Reducción de Funcionalidades: Se eliminaron características secundarias, manteniendo solo lectura de sensores y control básico.
2. Optimización Extrema: Se reescribió código para minimizar uso de memoria, eliminando buffers innecesarios.
3. Partición de Tareas: Se intentó dividir funcionalidades entre los dos núcleos del ESP32.

Ninguna de estas soluciones proporcionó estabilidad a largo plazo. El sistema invariablemente fallaba después de períodos variables de operación, requiriendo intervención manual.

5.3. Experiencia de Usuario y Estabilidad

- **Selección de Hardware:** El ESP32, aunque potente para aplicaciones simples, tiene limitaciones significativas para sistemas complejos que requieren múltiples protocolos de comunicación simultáneos en MicroPython.
- **MicroPython vs. ESP-IDF:** MicroPython ofrece desarrollo rápido pero sacrifica eficiencia de memoria y control de bajo nivel que hubieran sido esenciales para este proyecto.
- **Pruebas Tempranas:** Se subestimó la importancia de pruebas de estrés y carga máxima en etapas tempranas del desarrollo.

6. CONCLUSIONES

El proyecto "Sistema Autónomo de Gestión Hídrica para Horticultura Doméstica" representó un ejercicio valioso en el diseño de sistemas IoT complejos, aunque no logró implementarse completamente debido a limitaciones técnicas insuperables con la plataforma hardware-selección de software elegida.

6.1. Logros Parciales

A pesar del fracaso en la implementación completa, se lograron avances significativos:

1. **Diseño Arquitectónico:** Se desarrolló una arquitectura de sistema robusta y bien documentada.
2. **Integración Hardware:** Se verificó el funcionamiento individual de todos los componentes sensoriales y actuadores.
3. **Dashboard Funcional:** Se implementó exitosamente el dashboard Node-RED con todas las funcionalidades planeadas.
4. **Protocolos Individuales:** Tanto MQTT como BLE funcionaron correctamente cuando se probaron de forma aislada intercalada en el tiempo.

6.2. Limitaciones Críticas Identificadas

Hardware Inadecuado: El ESP32 con MicroPython demostró ser insuficiente para aplicaciones que requieren múltiples servicios de red concurrentes junto con procesamiento de sensores en tiempo real.

Compromisos de MicroPython: La facilidad de desarrollo de MicroPython tiene como contrapartida limitaciones severas en gestión de memoria y eficiencia que resultaron determinantes.

Complejidad Subestimada: La integración de diferentes tipos diferentes de sensores, dos actuadores y dos protocolos inalámbricos resultó ser más demandante de lo anticipado.

6.3. Recomendaciones para Implementaciones Futuras

Basado en las lecciones aprendidas, se recomienda:

1. **Cambio de Plataforma:** Utilizar el ESP32 programado con ESP-IDF (Framework de Desarrollo Oficial en C) en lugar de MicroPython, o considerar microcontroladores con mayor memoria como el ESP32-S3 con PSRAM.
2. **Arquitectura Distribuida:** Dividir el sistema en dos nodos: uno dedicado a adquisición de datos y control local, y otro dedicado a comunicaciones.
3. **Protocolo Único:** Optar por un solo protocolo de comunicación (por ejemplo, solo MQTT sobre Wi-Fi) eliminando BLE, o implementar BLE sólo para configuración inicial.
4. **Pruebas de Factibilidad:** Realizar pruebas de concepto rigurosas sobre limitaciones de memoria y procesamiento antes del desarrollo completo.

6.4. Valor Educativo

Aunque técnicamente no exitoso, el proyecto proporcionó un aprendizaje invaluable sobre:

- Las limitaciones reales de las plataformas IoT populares
- La importancia de la planificación de recursos en sistemas embebidos
- Los compromisos (trade-offs) entre facilidad de desarrollo y rendimiento
- Metodologías de diagnóstico de problemas complejos en sistemas embebidos

Este proyecto demuestra que incluso los fracasos técnicos, cuando son documentados y analizados rigurosamente, contribuyen significativamente a la formación de ingenieros capaces de tomar decisiones informadas en futuros desarrollos.

7. REFERENCIAS

Bonilla Hernández, P. L., Ruíz Menéndez, N. M., Chacha German, N. M., Tinoco Apolo, D. J., Agualongo Gavilanes, J. M., & Calle Andrade, J. D. (2025). Aprendizaje Basado en Proyectos con Internet de las Cosas (IoT) para la Creación de Prototipos Escolares que Solucionen Problemas Cotidianos. *Revista Multidisciplinar De Estudios Generales*, 4(2), 216–230. <https://doi.org/10.70577/reg.v4i2.91>

Digi International. (s.f.). ¿Cómo se comunican los dispositivos de IoT?. Digi.com. Recuperado el 9 de diciembre de 2025, de <https://es.digi.com/blog/post/how-do-iot-devices-communicate>

Finer, D. (s.f.). El papel clave del IoT en la eficiencia energética. Yoigo Luz y Gas. Recuperado el 9 de diciembre de 2025, de <https://www.yoigoluzygas.com/blog/papel-clave-del-iot-eficiencia-energetica/>

Montes, C. (2025, 17 de noviembre). Cómo IoT ayuda a mejorar la eficiencia energética. Beneficios. Alai Secure. Recuperado el 9 de diciembre de 2025, de <https://www.alaisecure.com/blog/como-iot-ayuda-a-mejorar-la-eficiencia-energetica-beneficios/>

Espressif Systems. (2023). *ESP32 Technical Reference Manual*. Espressif Systems. https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf

MicroPython Documentation. (2024). *Memory Management*. MicroPython. <https://docs.micropython.org/en/latest/reference/constrained.html>

Stack Overflow. (2024). *ESP32 MQTT and BLE coexistence issues*. Discusiones técnicas recopiladas sobre conflictos de recursos. <https://stackoverflow.com/questions/tagged/esp32+micropython+ble+mqtt>

Janjic, A. (2022). *Memory limitations in MicroPython on ESP32*. *Embedded Systems Journal*, 15(3), 45-52.

Rodríguez, M. (2023). *Protocol Coexistence in IoT Devices: Challenges and Solutions*. IEEE IoT Newsletter. <https://iot.ieee.org/newsletter>